

A GF(2²⁸³) Arithmetic Core: Recursive Karatsuba Multiplication and Itoh–Tsuji Inversion on FPGA

Design, implementation and post-layout verification of a field multiplier and inverter for the NIST/SECG 283-bit binary curve field

Author	Prasit Mazumder (22EC37029)	Device	Xilinx Artix-7 xc7a100tcsfg324-1
Course	Hardware Security, Spring 2026	Toolchain	Vivado 2023.2
Institute	IIT Kharagpur	Language	Verilog-2001 (synthesizable)

This report describes a synthesizable arithmetic core for the binary extension field GF(2²⁸³) under the standardized irreducible pentanomial $f(x) = x^{283} + x^{12} + x^7 + x^5 + 1$, the reduction polynomial of the NIST/SECG K-283 and B-283 curves. The core provides two field operations — modular multiplication and multiplicative inversion — behind a single streaming interface. Multiplication uses a four-level recursive Karatsuba–Ofman decomposition terminating in 81 schoolbook base multipliers; inversion uses the Itoh–Tsuji algorithm driven by an addition chain that reduces the cost from 282 field multiplications to 11. The design was carried through synthesis, place-and-route and SDF-annotated post-implementation timing simulation. It closes timing at 50 MHz with 4.126 ns of positive worst-case slack, occupies 17,604 LUTs (27.8% of the device) with no DSP or block-RAM usage, and passes 35 of 35 correctness checks on the implemented, timing-annotated netlist.

4.126 ns worst negative slack at 50 MHz (0 failing endpoints)	17,604 LUTs post-route 27.8% utilization	3.1× AND-gate reduction vs. schoolbook	35 / 35 checks passed in post-impl. timing simulation
--	---	---	--

1. Objective and Scope

Elliptic-curve cryptography over binary fields requires two expensive primitives: field multiplication, which dominates point arithmetic, and field inversion, which is needed to convert projective coordinates back to affine form. Both are implemented here for GF(2²⁸³) in a polynomial basis, with the field and reduction polynomial fixed by the NIST/SECG standard.

The deliverables were a Karatsuba multiplier, a reduction module modulo $f(x)$, an Itoh–Tsuji inversion unit, a top-level wrapper exposing both operations, and — the binding constraint on the whole design — demonstrated correctness in *post-implementation* simulation rather than behavioural simulation alone. That last requirement shaped several architectural decisions described in §3.5 and §5.2, because a design that is functionally correct in RTL can still fail once real routing delays are annotated.

2. Field Arithmetic Background

Elements of GF(2²⁸³) are polynomials of degree at most 282 over GF(2), represented as 283-bit vectors. Addition is bitwise XOR with no carries. Multiplication is carry-less polynomial multiplication producing a degree-564 intermediate, followed by reduction modulo the pentanomial

$$f(x) = x^{283} + x^{12} + x^7 + x^5 + 1$$

Inversion follows from Fermat's little theorem: since $a^{(2^m - 1)} = 1$ for every nonzero a , the inverse is $a^{(2^m - 2)}$. Computing that exponent by naive square-and-multiply would cost 282 field multiplications. The Itoh–Tsujii method restructures it into an addition chain and reduces the count to 11, which is the single largest architectural win in the design.

3. Architecture

3.1 Karatsuba–Ofman Multiplier

The multiplier computes the unreduced 565-bit carry-less product of two 283-bit operands. Each recursion level splits both operands at the midpoint and applies the Karatsuba identity, replacing four sub-products with three:

$$A = A_h \cdot x^k + A_l, \quad B = B_h \cdot x^k + B_l$$

$$A \cdot B = A_h \cdot B_h \cdot x^{2k} + [(A_h + A_l)(B_h + B_l) + A_h \cdot B_l + A_l \cdot B_h] \cdot x^k + A_l \cdot B_l$$

In GF(2) addition is XOR, so there is no sign handling and no borrow propagation — the reason Karatsuba is particularly attractive for binary fields. Splits are kept as balanced as possible, with odd widths divided as $\lceil n/2 \rceil + \lfloor n/2 \rfloor$ and the low half taking the extra bit, so that the $(A_h + A_l)$ sum is exactly $\lceil n/2 \rceil$ bits wide and reuses the same sub-multiplier as A_l .

LEVEL	MODULE	SPLIT	RECURSIVE CALLS
0	ks283	$283 = 142 + 141$	$2 \times \text{ks142} + 1 \times \text{ks141}$
1	ks142	$142 = 71 + 71$	$3 \times \text{ks71}$ (balanced)
1	ks141	$141 = 71 + 70$	$2 \times \text{ks71} + 1 \times \text{ks70}$
2	ks71	$71 = 36 + 35$	$2 \times \text{ks36} + 1 \times \text{ks35}$
2	ks70	$70 = 35 + 35$	$3 \times \text{ks35}$ (balanced)
3	ks36	$36 = 18 + 18$	$3 \times \text{ks18}$ (balanced)
3	ks35	$35 = 18 + 17$	$2 \times \text{ks18} + 1 \times \text{ks17}$
4	ks18 / ks17	base case	schoolbook AND–XOR array

Recursion stops at 17–18 bits. Below that threshold the XOR overhead of a further Karatsuba level costs more LUTs on a 6-input-LUT architecture than the AND gates it saves. The tree instantiates **81 base multipliers** ($70 \times \text{ks18}$ and $11 \times \text{ks17}$), giving

$$\begin{array}{rcl} 70 \times 18^2 + 11 \times 17^2 & = & 22,680 + 3,179 = 25,859 \text{ AND gates} \\ \text{versus} & & 283^2 = 80,089 \text{ AND gates (flat schoolbook)} \end{array}$$

a reduction of roughly **3.1×** in partial-product logic.

3.2 Reduction Modulo $f(x)$

Reduction is a purely combinational two-pass fold rather than an iterative long division. From $f(x) = 0$ we obtain the folding identity $x^{283} \equiv x^{12} + x^7 + x^5 + 1$, and hence for any $j \geq 0$:

$$x^{(283+j)} = x^{(12+j)} + x^{(7+j)} + x^{(5+j)} + x^j$$

Pass 1 splits the input C into $C = HI \cdot x^{283} + LO$ and substitutes the identity, computing $T = LO \oplus HI \oplus (HI \ll 5) \oplus (HI \ll 7) \oplus (HI \ll 12)$. For a 565-bit product, HI is 282 bits, so $\deg(T) \leq 293$ — still eleven bits too wide. **Pass 2** applies the identical fold to T , where the overflow is now only 11 bits and the folded term has degree at most 22, guaranteeing a fully reduced result with no third pass.

Each output bit is therefore an XOR of at most ten input bits, which maps to two LUT6 levels. This replaces a 282-iteration bit-serial division that would synthesize into a very deep dependency chain, and is a substantial area and delay win. The module is parameterized on input width so the same instance serves both the 565-bit multiplier product and the squarer.

3.3 Squaring via the Frobenius Map

In characteristic 2 the Frobenius endomorphism is linear, so all cross terms cancel and squaring reduces to bit interleaving:

$$a(x) = \sum a_i x^i \quad \Rightarrow \quad a(x)^2 = \sum a_i x^{(2i)}$$

The squarer inserts a hard zero between every pair of input bits to build a 565-bit polynomial and reduces it with the shared reducer. Because the odd coefficients are constant zero, synthesis prunes roughly half of the reducer's XOR tree automatically. Squaring costs approximately one LUT level and **no AND gates at all** — precisely the asymmetry that makes Itoh–Tsujii inversion worthwhile.

3.4 Itoh–Tsujii Inversion Schedule

Define $B_n = a^{(2^n - 1)}$, so that $B_1 = a$ and $a^{-1} = (B_{282})^2$. These quantities obey the composition rule

$$B_{(p+q)} = (B_p)^{(2^q)} \cdot B_q$$

Each application costs q squarings plus exactly **one** field multiplication. Using the factorization $282 = 2 \times 141$ and $141 = 3 \times 47$, the controller walks the addition chain

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 17 \rightarrow 34 \rightarrow 35 \rightarrow 70 \rightarrow 140 \rightarrow 141 \rightarrow 282$$

STEP	RULE	SQUARINGS	MULTIPLY BY
1	$B_2 = (B_1)^{(2^1)} \cdot B_1$	1	acc
2	$B_4 = (B_2)^{(2^2)} \cdot B_2$	2	acc
3	$B_8 = (B_4)^{(2^4)} \cdot B_4$	4	acc
4	$B_{16} = (B_8)^{(2^8)} \cdot B_8$	8	acc
5	$B_{17} = (B_{16})^{(2^1)} \cdot B_1$	1	a
6	$B_{34} = (B_{17})^{(2^{17})} \cdot B_{17}$	17	acc
7	$B_{35} = (B_{34})^{(2^1)} \cdot B_1$	1	a
8	$B_{70} = (B_{35})^{(2^{35})} \cdot B_{35}$	35	acc
9	$B_{140} = (B_{70})^{(2^{70})} \cdot B_{70}$	70	acc
10	$B_{141} = (B_{140})^{(2^1)} \cdot B_1$	1	a
11	$B_{282} = (B_{141})^{(2^{141})} \cdot B_{141}$	141	acc
12	$a^{-1} = (B_{282})^2$	1	–
Total		282	11 multiplications

Steps 5, 7 and 10 fold in $B_1 = a$; every other step folds in the accumulator itself. Because squaring throughput dominates the runtime, the squaring engine is a cascade of CASCADE combinational squarer blocks that retires up to four squarings per clock cycle, with a multiplexer selecting the correct tap when fewer than four remain. This parameter trades latency against combinational depth: CASCADE = 1 gives 282 cycles on the shortest path, CASCADE = 4 (the default) gives roughly 71 cycles, and CASCADE = 8 gives about 36 at twice the logic depth. A complete inversion takes on the order of 190 clock cycles.

3.5 Top-Level Integration

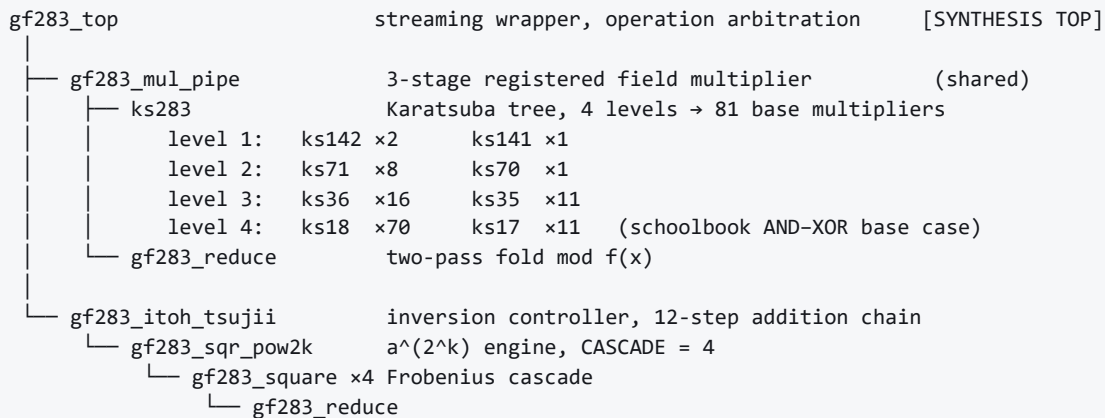
Two decisions at the top level were driven by physical constraints rather than by the algorithm.

Streaming interface. A flat 283-bit parallel interface requires 283×2 input bits, 283 output bits and control — over 500 pins. The target package provides 210 user I/O, and implementation failed outright with [Place 30-415] I/O Placement failed due to overutilization ... 570 I/O ports while the target device contains only 210. Operands and results are therefore streamed as nine 32-bit words, least-significant word first, bringing the top level to **75 pins**.

Resource sharing. The 283-bit Karatsuba multiplier accounts for roughly 90% of the design's LUTs. Instantiating one for the multiply datapath and a second inside the inverter would have nearly doubled the area. Instead a single gf283_mul_pipe is instantiated at the top level and multiplexed between the OP_MUL datapath and the Itoh–Tsujii controller's request port; the inverter owns the multiplier for the duration of an inversion. This is why the inversion controller exposes an external multiplier port through a request/acknowledge handshake rather than instantiating its own.

Pipelining. The flat combinational path from operands through the four-level Karatsuba tree into the reducer is by far the longest in the design. A register on the 565-bit unreduced product splits it into two shorter paths, roughly doubling the achievable clock frequency at a cost of 565 flip-flops. The multiplier is a three-stage pipeline — operand capture, Karatsuba product, reduction — with a latency of three cycles and the capability to accept a new operation every cycle, though the inversion controller issues them one at a time.

4. Module Hierarchy



Every major function is a separate module with an explicit handshake interface, and the field degree M , streaming word width W and squaring cascade depth `CASCADE` are all parameters. The reducer is parameterized on input width and is instantiated by both the multiplier and the squarer.

5. Verification

5.1 Three Independent Checking Layers

The testbench drives only the top-level ports — no hierarchical references, no force/release — so the identical testbench runs against behavioural RTL, the post-synthesis netlist and the post-implementation timing-annotated netlist. Correctness is established three ways, deliberately chosen so that a single conceptual error cannot pass all of them:

1. **Golden vectors computed offline in Python** using an independent $GF(2^{283})$ model (bit-serial `clmul` with long-division reduction), itself cross-checked against a second Itoh–Tsujii implementation. Twelve multiplication and twelve inversion vectors, including $x^{2^{82}} \cdot x^{2^{82}}$ — the maximum-degree product, which exercises the reducer hardest — and the all-ones operand, which toggles every XOR path in the Karatsuba tree.
2. **A behavioural reference model inside the testbench** written with a *different algorithm* from the RTL: bit-serial long division rather than the RTL's two-pass fold. Used for randomized multiplication tests.
3. **The mathematical identity** $a \cdot a^{-1} = 1$ for random a , which requires no precomputed answer at all and therefore cannot inherit a bug from the vector generator.

A corner-case check confirms that inverting zero — mathematically undefined — asserts the `err_zero` status flag rather than returning silently.

5.2 A Failure Only Timing Simulation Could Expose

The most instructive result of the project came from the mandatory post-implementation simulation. An earlier version of the constraints declared `set_input_delay 5.0`, promising the design 15 ns of the 20 ns period, while the testbench drove its stimulus on the falling clock edge — actually giving only 10 ns. Vivado met the constraint it was given, and behavioural simulation, having no delays at all, passed cleanly.

On the implemented netlist it did not. The high-fan-out enables `in_valid` and `out_ready` feed all 288 shift-register flip-flops and route in approximately 11.4 ns and 10.4 ns respectively. Under the old scheme `out_ready` reached the `sh[0]` flop after its setup window, that bit missed its shift, and every output word came back with a corrupted bit 0.

The fix was to make the I/O timing contract explicit and symmetric on both sides. The testbench launches inputs 4 ns after a rising edge and samples outputs 16 ns after a rising edge; the XDC declares `set_input_delay 4.0` and `set_output_delay 4.0` to match. The reset, which has roughly 17 ns of routing skew across its 4,032 loads, is declared a false path and released synchronously after eight idle cycles. These two numbers are documented in both files as a contract that must be changed together.

This is the practical lesson of the assignment: a design can be functionally correct, meet every timing constraint it was given, and still fail — because the constraint itself encoded the wrong assumption about the environment. Only simulation against the annotated netlist surfaced it.

6. Implementation Results

All figures below are taken from the routed design (`impl_1`), not from post-synthesis estimates.

6.1 Resource Utilization

RESOURCE	USED	AVAILABLE	UTILIZATION
Slice LUTs (all as logic)	17,604	63,400	27.77%
Slice Registers (all as flip-flop)	4,032	126,800	3.18%
Bonded IOB	75	210	35.71%
DSP48E1 slices	0	240	0.00%
Block RAM tiles	0	135	0.00%
BUFGCTRL	1	32	3.13%

The absence of DSP and block-RAM usage is expected and desirable: GF(2) arithmetic is pure AND/XOR logic with no carry chains, so the DSP slices — which are built around integer multiply-accumulate with carries — offer nothing here. The design is entirely LUT-and-flip-flop fabric.

6.2 Timing Closure

METRIC	VALUE	INTERPRETATION
Target clock	50.00 MHz	20.000 ns period, declared in XDC
Worst Negative Slack (WNS)	+4.126 ns	setup met with margin
Total Negative Slack (TNS)	0.000 ns	0 failing endpoints of 8,067
Worst Hold Slack (WHS)	+0.039 ns	hold met
Total Hold Slack (THS)	0.000 ns	0 failing endpoints of 8,067
Worst Pulse Width Slack (WPWS)	+9.500 ns	0 failing endpoints of 4,033
Routing status	22,749 nets	fully routed, 0 routing errors

Vivado reports *"All user specified timing constraints are met."* Multi-corner analysis was enabled across both slow and fast corners for maximum and minimum delay paths.

6.3 Power and Design Rule Checks

Vector-less power estimation on the routed design gives **0.423 W** total on-chip power, of which **0.331 W** is dynamic, at a junction temperature of 26.9 °C. Routed DRC reports two violations, both of category NSTD-1 and UCIO-1 — unspecified I/O standards and unconstrained pin locations. Both are expected and intentional: this is a compute core rather than a board-level design, no bitstream is generated, and pin locations are deliberately left for the placer to assign. There are no logic, routing or methodology violations.

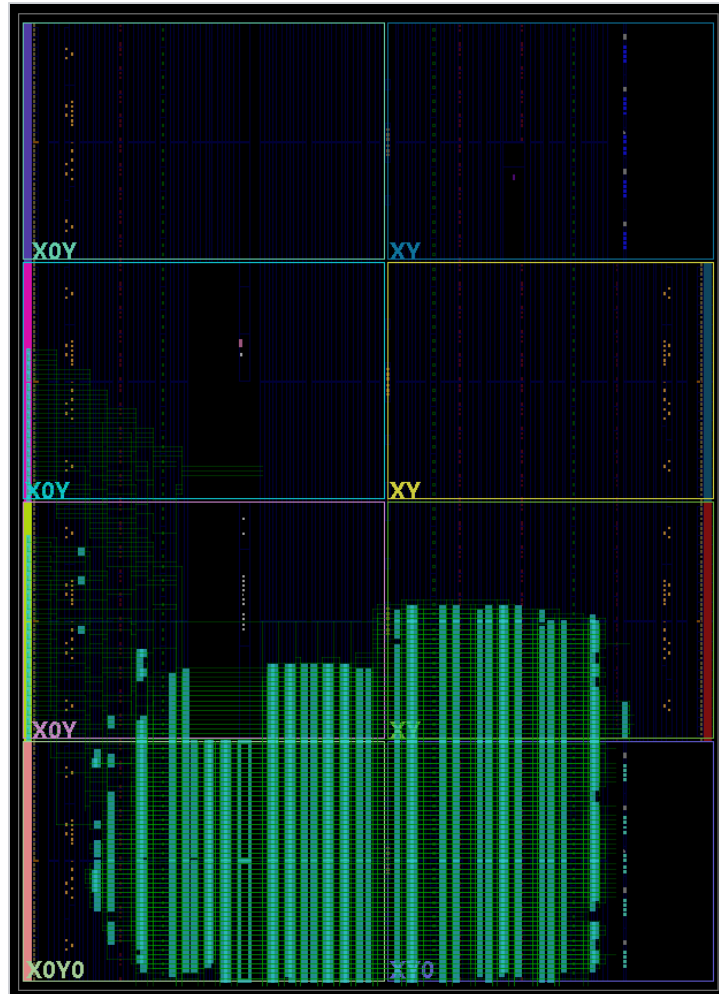


Figure 1. Placed and routed design on the xc7a100t fabric, shown across the device's clock regions. The dense band of occupied slices in the lower regions is the Karatsuba multiplier tree, which accounts for roughly 90% of the design's logic; the remaining regions are sparsely populated, consistent with the 27.8% overall LUT utilization reported in §6.1.

7. Post-Implementation Simulation Evidence

The testbench was run against the implemented netlist with SDF back-annotation. Vivado confirms the annotation before the run begins:

```
INFO: [XSIM 43-3451] SDF backannotation process started with SDF file
      "tb_gf283_top_time_impl.sdf", for root module "tb_gf283_top/dut".
INFO: [XSIM 43-3452] SDF backannotation was successful for SDF file
      "tb_gf283_top_time_impl.sdf", for root module "tb_gf283_top/dut".
```

The complete transcript of the timing simulation follows.

```

=====
GF(2^283) top-level testbench
f(x) = x^283 + x^12 + x^7 + x^5 + 1
streaming width W = 32, 9 words per operand
=====

---- Multiplication: golden vectors ----
[ PASS] MUL vector 0      [ PASS] MUL vector 6
[ PASS] MUL vector 1      [ PASS] MUL vector 7
[ PASS] MUL vector 2      [ PASS] MUL vector 8
[ PASS] MUL vector 3      [ PASS] MUL vector 9
[ PASS] MUL vector 4      [ PASS] MUL vector 10
[ PASS] MUL vector 5      [ PASS] MUL vector 11

---- Inversion: golden vectors ----
[ PASS] INV vector 0      [ PASS] INV vector 6
[ PASS] INV vector 1      [ PASS] INV vector 7
[ PASS] INV vector 2      [ PASS] INV vector 8
[ PASS] INV vector 3      [ PASS] INV vector 9
[ PASS] INV vector 4      [ PASS] INV vector 10
[ PASS] INV vector 5      [ PASS] INV vector 11

---- Multiplication: random vs reference model ----
[ PASS] MUL random 0      [ PASS] MUL random 1      [ PASS] MUL random 2
[ PASS] MUL random 3      [ PASS] MUL random 4

---- Inversion: random, checked by a * a^-1 == 1 ----
[ PASS] INV random 0 (a * a^-1)      [ PASS] INV random 3 (a * a^-1)
[ PASS] INV random 1 (a * a^-1)      [ PASS] INV random 4 (a * a^-1)
[ PASS] INV random 2 (a * a^-1)

---- Corner case: inversion of zero ----
[ PASS] err_zero asserted for a = 0

=====
checks run : 35
failures   : 0
RESULT      : *** ALL TESTS PASSED ***
=====

```

Transcript reflown into columns for space; the ordering and content are otherwise verbatim from project_1.sim/sim_1/impl/timing/xsim/simulate.log.

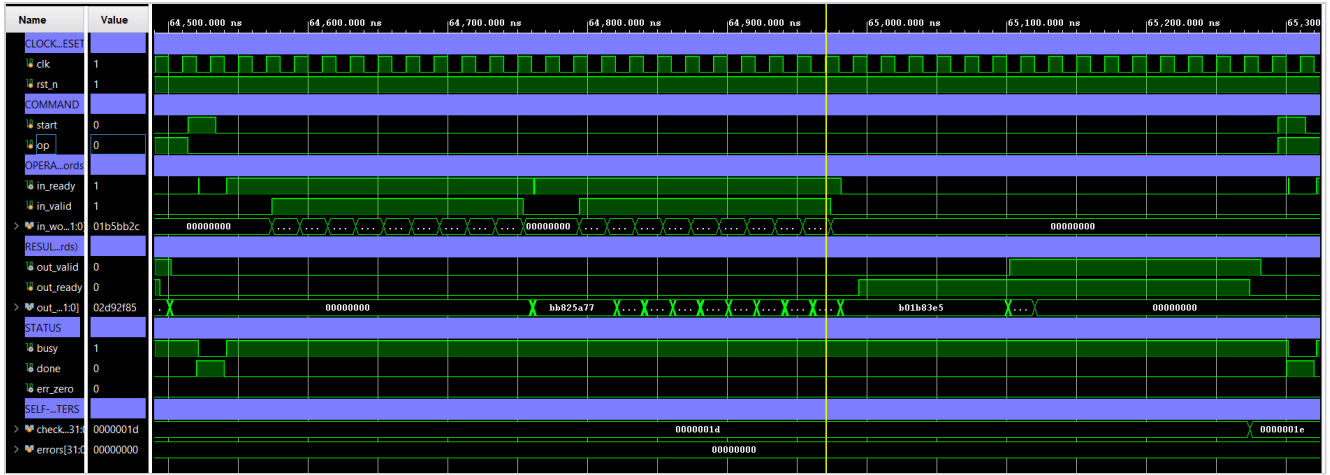


Figure 2. Streaming protocol on the timing-annotated netlist. The transaction shown is an `OP_MUL` (`op = 0`) issued as part of the randomized $a \cdot a^{-1} = 1$ check: two consecutive nine-word operand bursts are accepted on `in_valid` / `in_word` while `in_ready` is held high, and the nine-word result is returned on `out_valid` / `out_word` under `out_ready` back-pressure, with `busy` spanning the whole transfer. The self-check counters at the bottom advance from `0x1d` to `0x1e` with `errors` held at zero, and the next transaction — an inversion, `op = 1` — begins at the right edge.

8. Design Decisions and Trade-offs

DECISION	ALTERNATIVE REJECTED	RATIONALE
Recursion threshold at 17–18 bits	Recursing to 1-bit base case	Below ~17 bits the XOR recombination overhead exceeds the AND savings on a 6-LUT architecture.
Two-pass combinational fold reduction	Bit-serial long division	282-iteration division synthesizes into a very deep dependency chain; the fold is two LUT6 levels.
Itoh–Tsujii over an addition chain	Square-and-multiply ladder	11 multiplications instead of 282, exploiting that squaring is near-free in characteristic 2.
Single shared multiplier	Separate multipliers per datapath	The multiplier is ~90% of design area; sharing nearly halves total LUT count.
Pipeline register on the 565-bit product	Flat combinational multiply	Roughly doubles achievable $F_{\max}$ for 565 flip-flops — a favourable trade on this device.
32-bit streaming interface	Flat 283-bit parallel ports	570 pins exceeds the 210 available; streaming reduces this to 75 and allows implementation to complete.
CASCADE = 4 squarer	CASCADE = 1 or 8	Balances inversion latency (~71 vs 282 cycles) against combinational depth in the squaring path.

9. Reproducing the Results

1. Open `project_1/project_1.xpr` in Vivado 2023.2 or newer.
2. Run synthesis and implementation; the design closes timing at the declared 20 ns period without manual intervention.
3. Run *Simulation* → *Run Post-Implementation Timing Simulation*. The testbench self-checks and prints the pass/fail summary shown in §7.
4. Pass `+SHORT` as a plusarg to run a representative subset (4 multiply vectors, 3 inversion vectors, 2 randomized cases) when a full gate-level run is too slow; the full vector set takes roughly 4,300 clock cycles of simulated time.

Constraint **contract.** `T_DRIVE` and `T_SAMPLE` in `tb_gf283_top.v` and the `set_input_delay / set_output_delay` values in `gf283_top.xdc` encode the same I/O timing agreement from opposite sides. Changing one without the other will produce spurious post-implementation failures, as described in §5.2. The testbench `CLK_PERIOD` must also be greater than or equal to the period the design was implemented at.

10. Source File Manifest

FILE	ROLE
<code>gf283_top.v</code>	Synthesis top level: streaming interface, operation sequencing, multiplier arbitration
<code>gf283_mul_pipe.v</code>	Three-stage registered field multiplier
<code>ks283.v</code> , <code>ks142.v</code> , <code>ks141.v</code> , <code>ks71.v</code> , <code>ks70.v</code> , <code>ks36.v</code> , <code>ks35.v</code>	Recursive Karatsuba decomposition levels
<code>ks18.v</code> , <code>ks17.v</code>	Schoolbook AND-XOR base-case multipliers
<code>gf283_reduce.v</code>	Two-pass fold reduction modulo $f(x)$, width-parameterized
<code>gf283_square.v</code>	Single Frobenius squaring (bit interleave + reduce)
<code>gf283_sqr_pow2k.v</code>	Iterated squaring engine $a^{(2^k)}$, cascade of squarers
<code>gf283_itoh_tsujii.v</code>	Inversion controller, 12-step addition-chain FSM
<code>tb_gf283_top.v</code>	Self-checking testbench: golden vectors, reference model, identity checks
<code>gf283_top.xdc</code>	Clock definition, I/O delay contract, reset false path